Análisis de Caso

Servicios de bases de datos relacionales

Eva Durán

alkemy

Análisis de Caso

Servicios de Bases de Datos Relacionales

En este caso, aplicaremos los conceptos de bases de datos relacionales en un entorno cloud, usando el plan gratuito de AWS Academy (o la capa gratuita de AWS) para experimentar con la configuración y administración de una instancia de Amazon RDS de manera práctica.

Situación inicial 📍

Una pequeña startup de comercio electrónico llamada **ShopNow** está en etapa de crecimiento. Hasta el momento, han estado utilizando un servidor local para almacenar la información de productos, usuarios y pedidos, pero el volumen de datos va en aumento y se han detectado problemas de rendimiento y disponibilidad.

El equipo de TI propone migrar la base de datos a la nube para aprovechar ventajas como la alta disponibilidad y la facilidad de escalado. Dado que los costos son un factor crítico, deciden **usar la capa gratuita** de AWS, en especial con un **motor de base de datos sin licencias adicionales** (p. ej., MySQL o PostgreSQL). Además, la empresa desea contar con una estrategia de respaldos automáticos y un método sencillo de recuperación en caso de fallas.

Descripción del Caso 🔎

Asumes el papel de **Administrador de Bases de Datos** (DBA) responsable de la transición hacia un servicio gestionado de base de datos relacional. Tu tarea consiste en diseñar, crear y configurar una instancia de Amazon RDS con el plan gratuito, garantizando tanto la seguridad como la eficiencia en el acceso a los datos. Adicionalmente, tu misión será sentar las bases para un futuro escalamiento si el negocio crece y se incrementa el volumen de operaciones.

Deberás tomar decisiones sobre:

- El tipo de motor de base de datos (MySQL, PostgreSQL, etc.) dentro de los disponibles en AWS RDS.
- La implementación de respaldos automáticos y snapshots puntuales.
- El diseño de esquemas de tablas para la información de productos, usuarios y pedidos, asegurando la integridad referencial y la consistencia de los datos.

DESARROLLO 📄

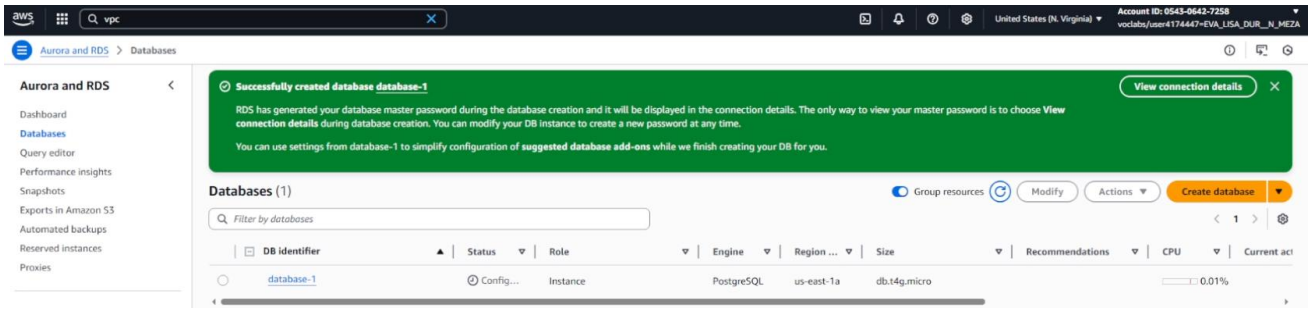
Migración de base de datos a Amazon RDS – Caso ShopNow

1. Elección del motor de base de datos en RDS

Para el caso de ShopNow se seleccionó PostgreSQL como motor de base de datos en Amazon RDS por las siguientes razones:

- **Familiaridad técnica**: es el motor que manejo, lo cual reduce los errores de configuración y agiliza el desarrollo.
- **Código abierto**: es gratuito y no implica licencias comerciales, lo cual lo hace ideal para este caso en el que la startup presenta un presupuesto limitado.
- **Confiabilidad y robustez**: Postgres está ampliamente reconocido por su cumplimiento de estándares ACID, lo que asegura integridad y consistencia de datos.
- **Compatibilidad con RDS Free Tier**: Es unos de los motores soportados en la capa gratuita de AWS, lo que se ajusta con el presupuesto limitado del caso.
- **Extensibilidad**: Permite el uso de funciones personalizadas, tipos de datos personalizados y soporte para JSON, ideal para necesidades futuras de ShopNow. Además, tiene muchas extensiones oficiales y de la comunidad que permiten ampliar la funcionalidad de la base de datos que podría ser útil para las necesidades actuales y/o futuras de la startup.

Instancia RDS creada:



El tamaño de la instancia corresponde a lo siguiente:

Escenario	Clase recomendada	vCPU / RAM (aprox.)	Cuándo usarla
Dev/pruebas (aprovechar capa gratuita)	**db.t4g.micro**	2 vCPU / **1 GiB**	Para desarrollo, QA, demos y cargas bajas. Coste mínimo.

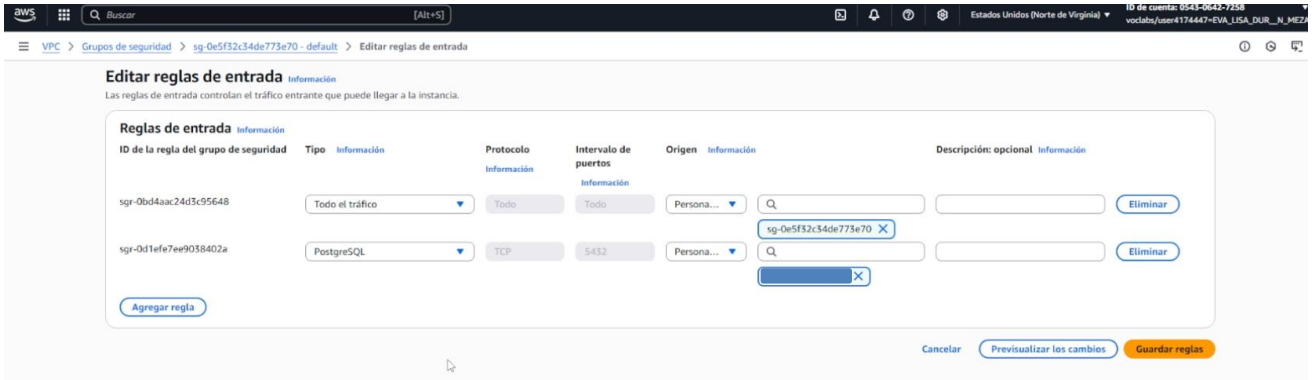
Se seleccionó Single-AZ y en cuanto a tipo de almacenamiento: gp3 (General Purpose SSD) con los valores por defecto.

2. Configuración de seguridad y accesibilidad

En esta sección se detallan los pasos de configuración de seguridad y accesibilidad, en particular para configurar los grupos de seguridad y de cifrado.

Configuración Grupo de seguridad:

Grupos de seguridad > nombre grupo de seguridad > Editar reglas de entrada



Se limitó el acceso a la instancia RDS:

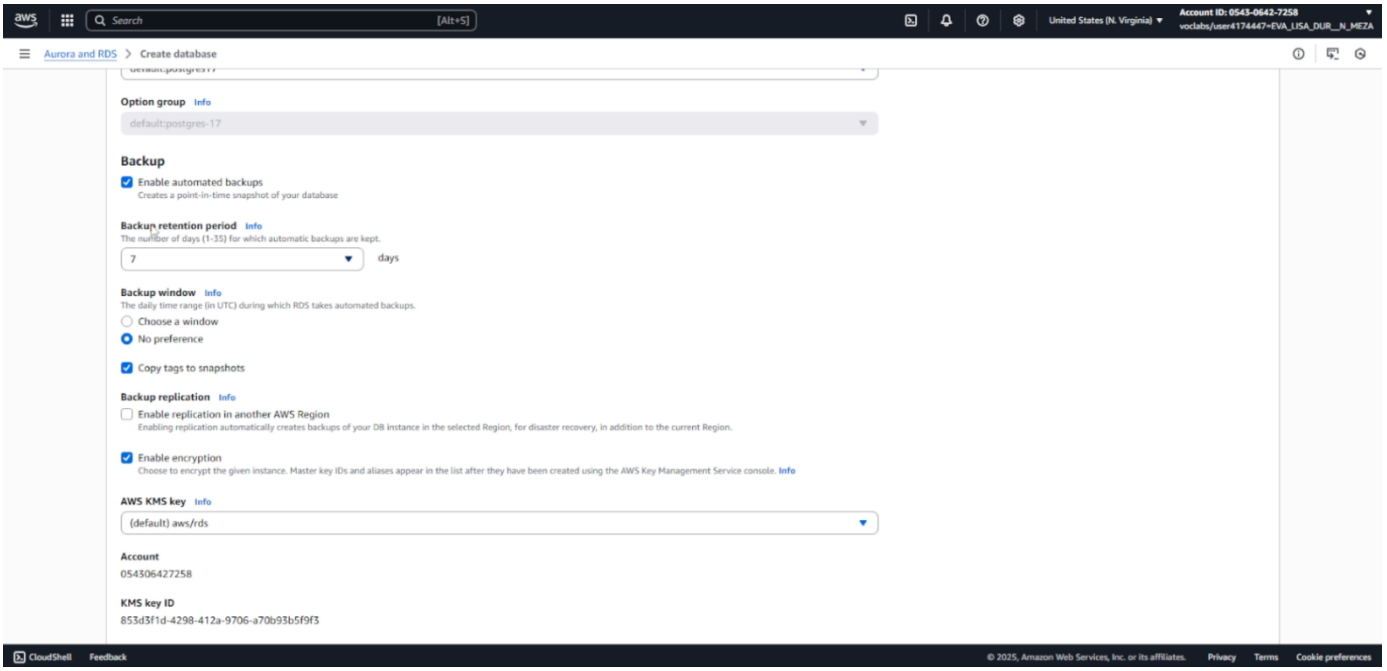
Regla 1: "Todo el tráfico" limitado solo al Grupo de seguridad (el por defecto, de la propia VPC).

Regla 2: Acceso al puerto PostgreSQL solo desde mi IP pública.

Con esto Nadie externo (fuera de mi IP y de las instancias en la VPC) puede conectarse.

Configuración de cifrado de datos:

El cifrado en reposo se puede configurar al momento de crear la base de datos:



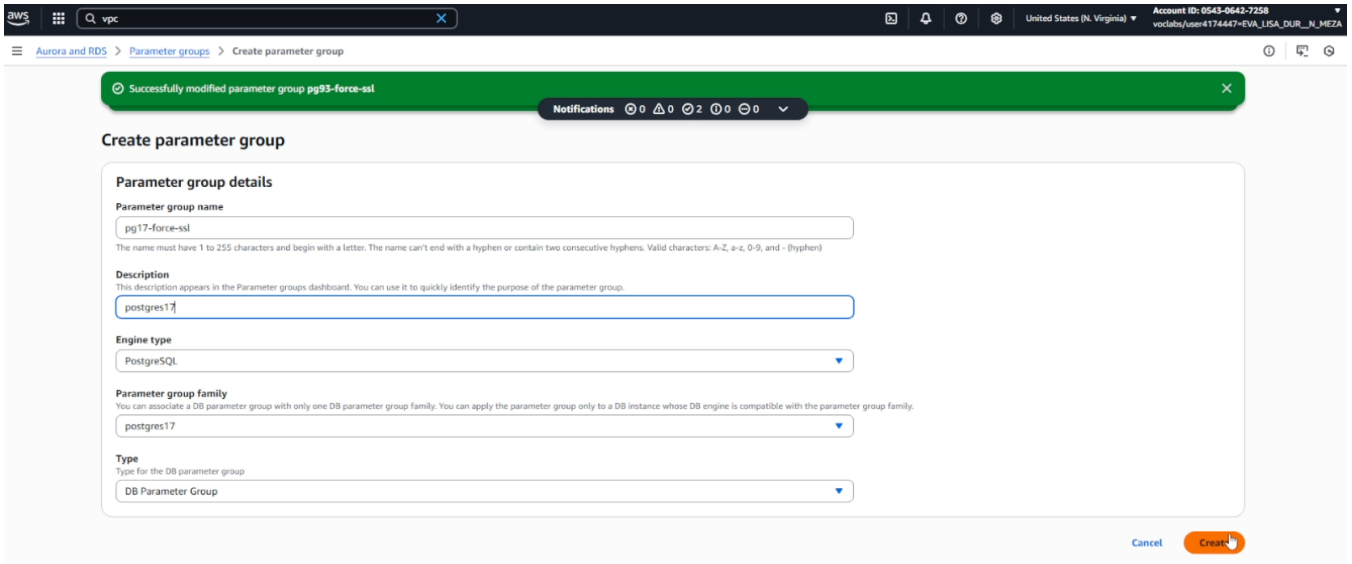
Hay que seleccionar la opción:

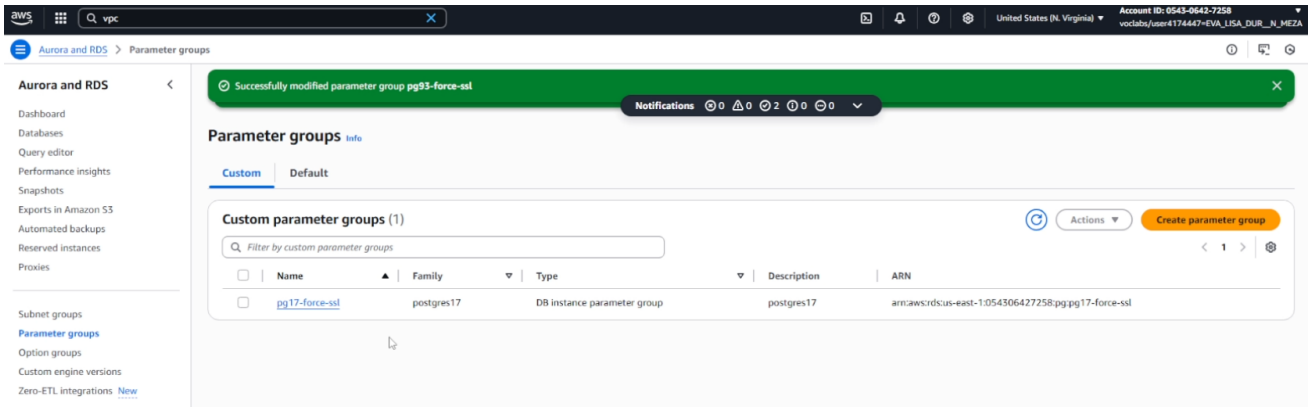
☑ Enable encryption (habilitar el cifrado)

Esto protege los datos cuanto están almacenados en el disco de RDS (incluye backups, snapshots y réplicas).

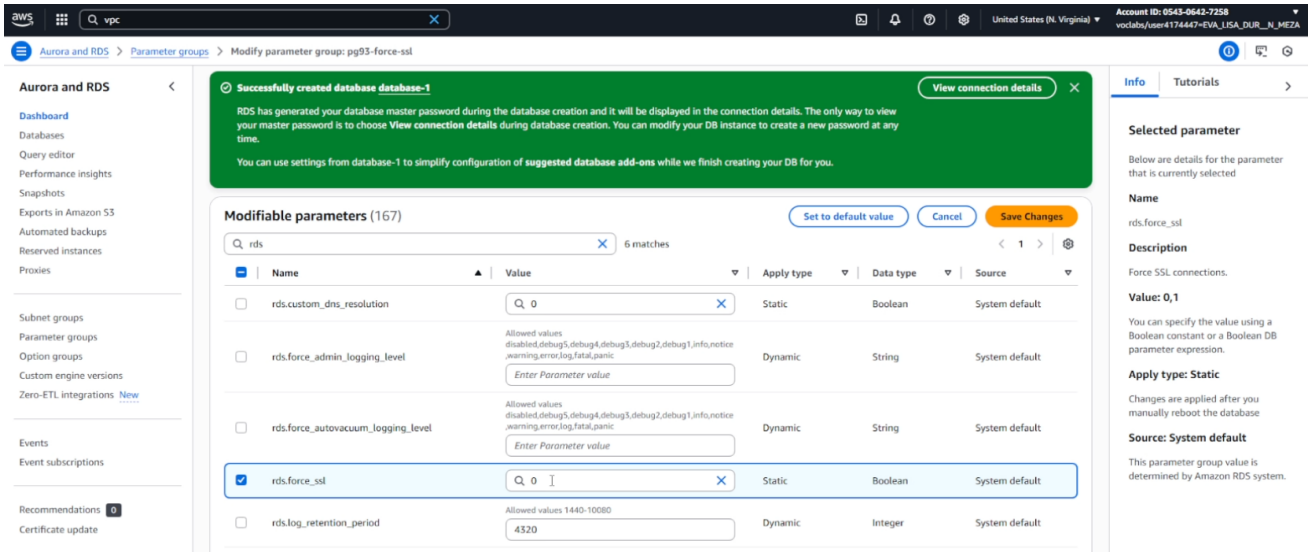
Para el cifrado en tránsito (TLS/SSL) se puede forzar que todas las conexiones usen SSL. Esto protege los datos mientras viajan entre la aplicación/cliente y la base de datos. Para esto hay que ir a Aurora y RDS > Parámetros de grupos > Crear parámetro de grupo:

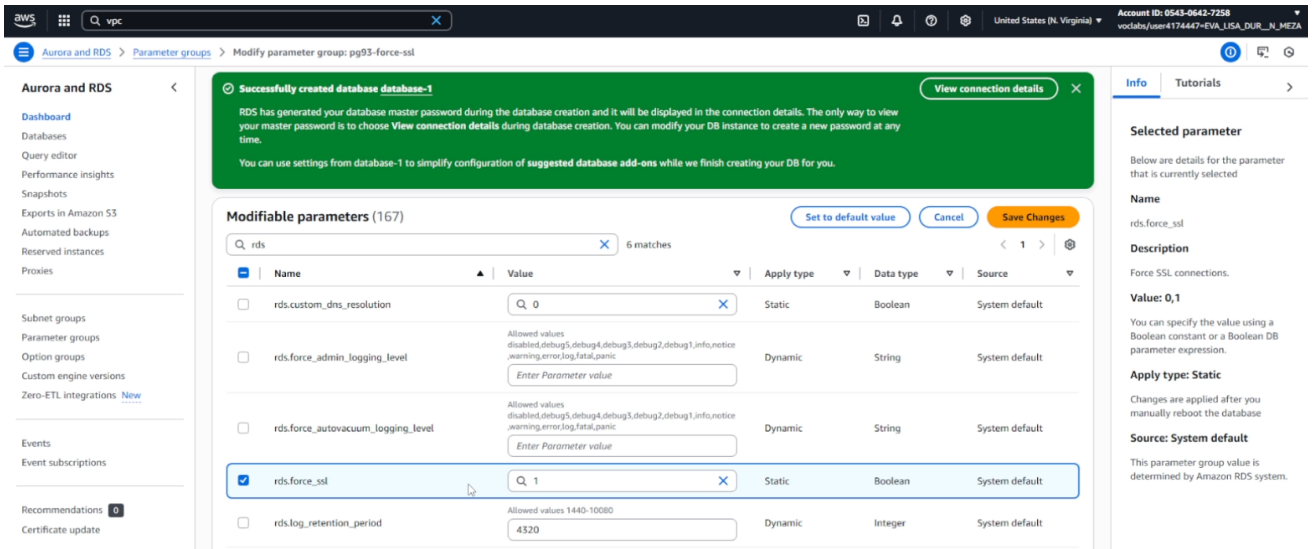
1. Crear un nuevo grupo para poder editarlo.
2. Rellenar los campos solicitados:
    a. Parameter group name: es un identificador único sin espacios, que empiece con letra y solo use letras, números y guiones (-). Si tu RDS es PostgreSQL 17, se recomienda nombrarlo como: pg17-force-ssl
    b. Description: Texto para recordar el propósito (ejemplo: Grupo de parámetros para forzar SSL en RDS PostgreSQL 17)
    c. Engine type: PostgreSQL
    d. Parameter group family: coincide con la versión, en este caso postgres17.



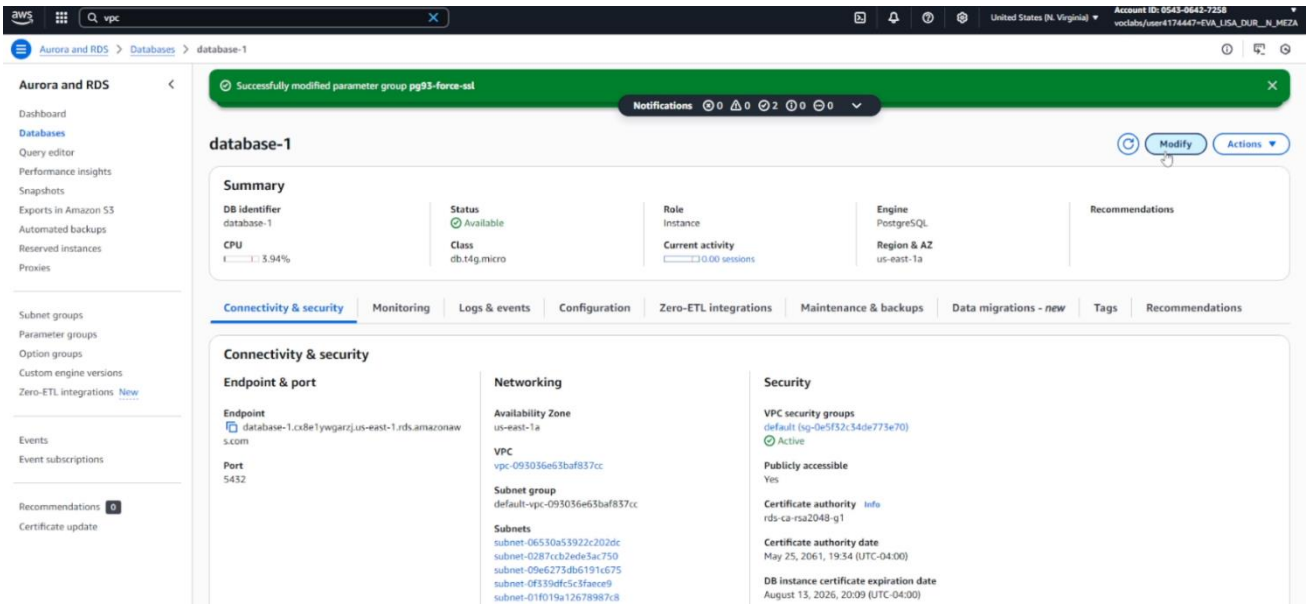


3. Una vez creado ir a: Aurora y RDS > Parámetros de grupos > pg17-force-ssl y buscar “rds.force_ssl” y cambiar de 0 a 1.

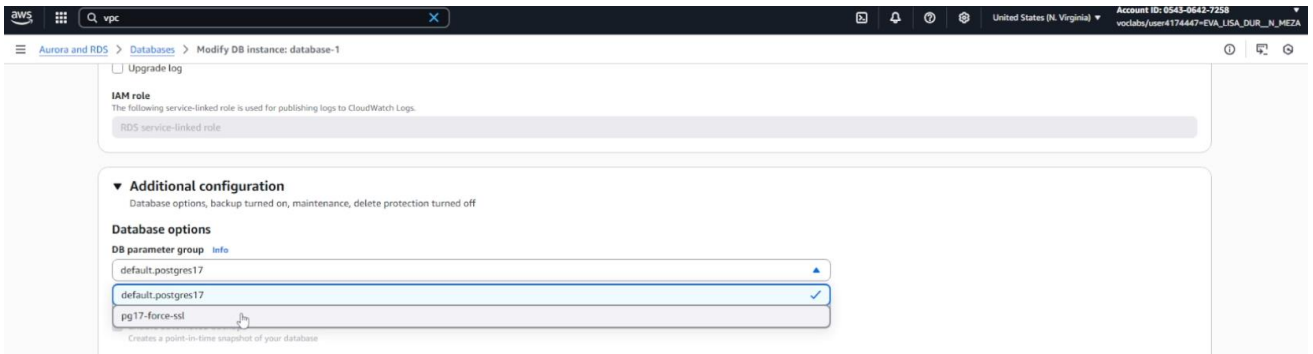




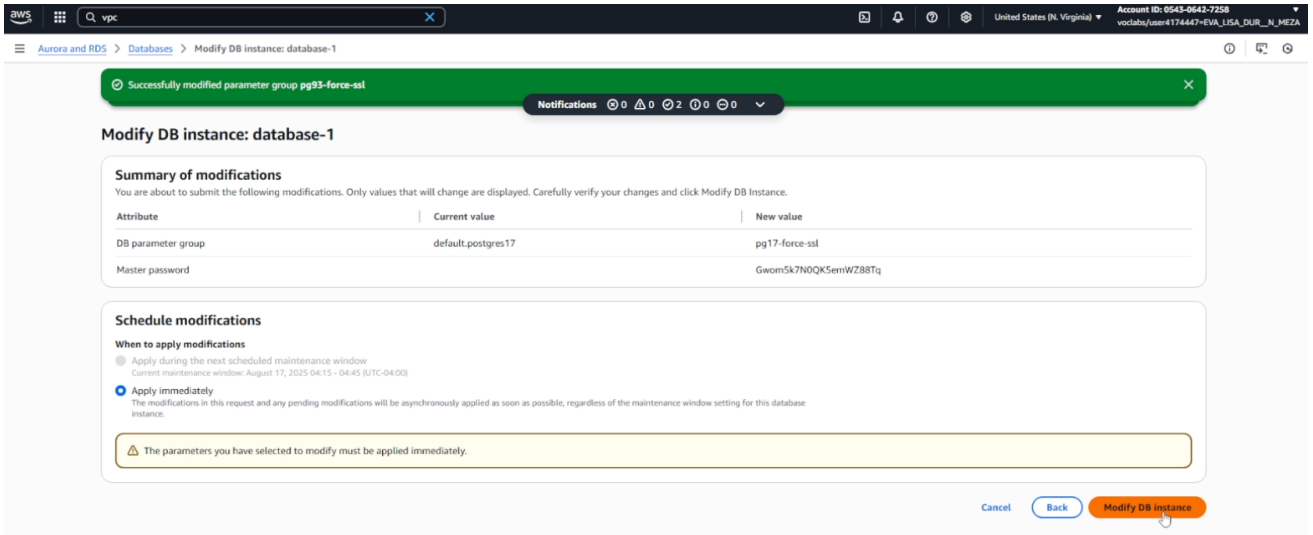
4. Guardar cambios.
5. Luego ir a la instancia RDS y hacer click en “Modify” (o Editar)



6. En la sección Additional configuration > Database option > DB parameter group: seleccionar pg17-force-ssl



7. Finalmente, modificar DB instance



3. Diseño del esquema y creación de tablas

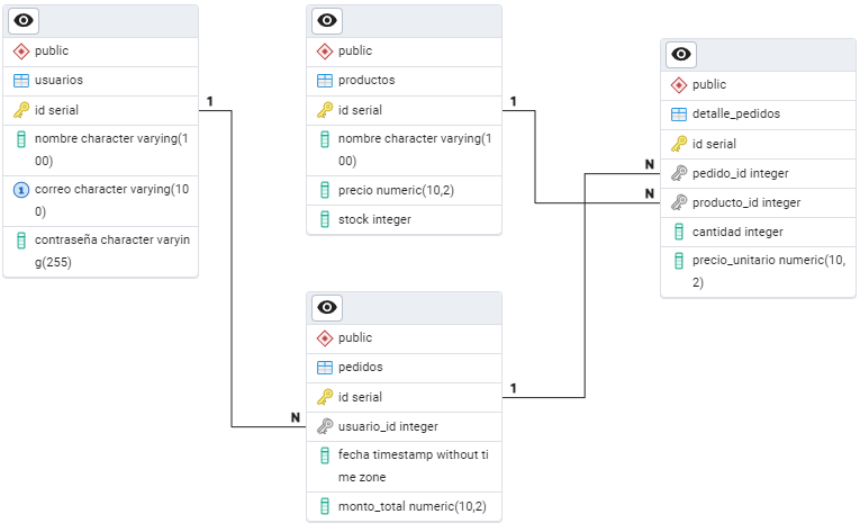
A continuación, se presenta el esquema básico para la creación de tablas junto con 5 registros de ejemplo por cada tabla.

```
-- Crear Base de datos
CREATE DATABASE ecommerce

-- Tabla de usuarios

CREATE TABLE usuarios (
    id SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    correo VARCHAR(100) UNIQUE NOT NULL,
    contraseña VARCHAR(255) NOT NULL
);

-- Tabla de productos
CREATE TABLE productos (
```

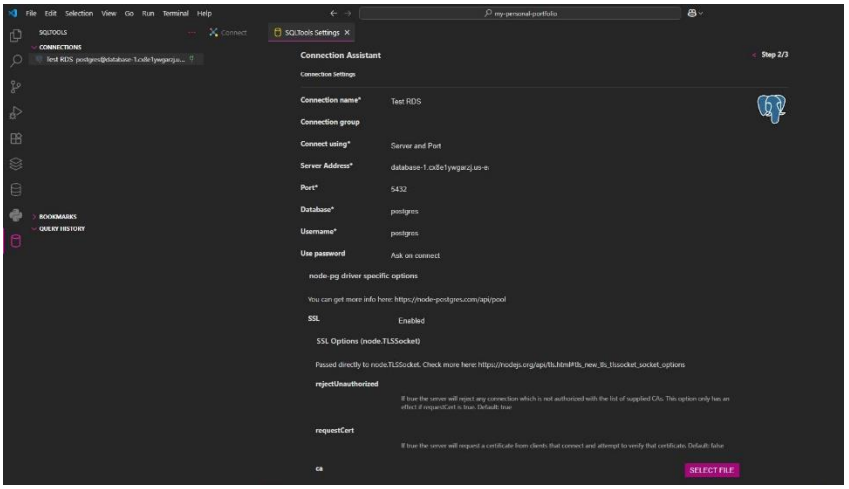
```sql
    id SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    precio NUMERIC(10,2) NOT NULL,
    stock INTEGER NOT NULL
);

-- Tabla de pedidos
CREATE TABLE pedidos (
    id SERIAL PRIMARY KEY,
    usuario_id INTEGER REFERENCES usuarios(id),
    fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    monto_total NUMERIC(10,2) NOT NULL
);

-- Tabla intermedia: detalle de productos en pedidos
CREATE TABLE detalle_pedidos (
    id SERIAL PRIMARY KEY,
    pedido_id INTEGER REFERENCES pedidos(id),
    producto_id INTEGER REFERENCES productos(id),
    cantidad INTEGER NOT NULL,
    precio_unitario NUMERIC(10,2) NOT NULL
);

-- Insertar datos de ejemplo
INSERT INTO usuarios (nombre, correo, contraseña) VALUES
('Ana Torres', 'ana@example.com', 'clave123'),
('Luis Pérez', 'luis@example.com', 'seguro456'),
('Claudia Ruiz', 'claudia@example.com', 'pass789'),
('Mario Soto', 'mario@example.com', 'm1234'),
('Valeria Díaz', 'valeria@example.com', 'vpass');

INSERT INTO productos (nombre, precio, stock) VALUES
('Mouse inalámbrico', 15.99, 50),
('Teclado mecánico', 39.99, 20),
('Monitor 24"', 119.90, 10),
('Laptop 14"', 479.99, 8),
('Auriculares Bluetooth', 25.50, 35);

INSERT INTO pedidos (usuario_id, fecha, monto_total) VALUES
(1, NOW(), 55.98),   -- 2 auriculares
(2, NOW(), 479.99),  -- 1 laptop
(3, NOW(), 135.89),  -- 1 teclado + 1 mouse
(4, NOW(), 25.50),   -- 1 auricular
(5, NOW(), 239.80);  -- 2 monitores

INSERT INTO detalle_pedidos (pedido_id, producto_id, cantidad, precio_unitario)
VALUES
(1, 5, 2, 25.50),   -- Pedido 1: 2 auriculares
(2, 4, 1, 479.99),  -- Pedido 2: 1 laptop
(3, 2, 1, 39.99),   -- Pedido 3: 1 teclado
(3, 1, 1, 15.99),   -- Pedido 3: 1 mouse
(5, 3, 2, 119.90);  -- Pedido 5: 2 monitores
```

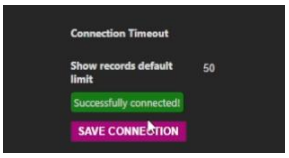
BBDD-ShopNow.sql

Además, se adjunta el diagrama ERD del caso:

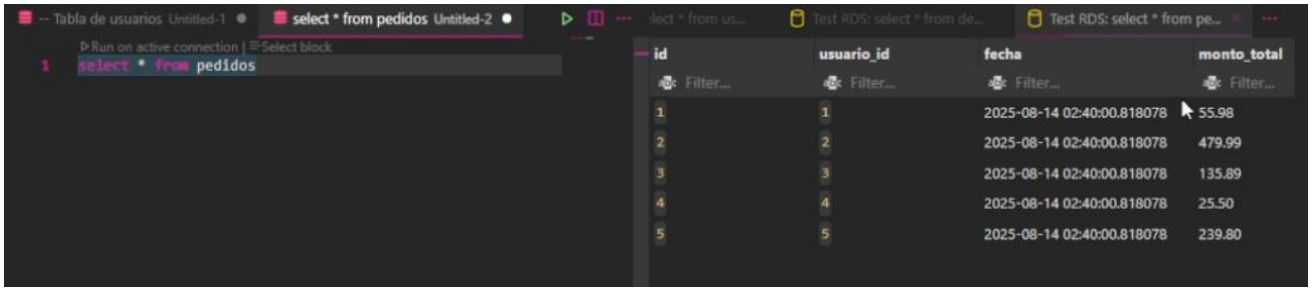


Conexión a la instancia de Amazon RDS a través de VS Code utilizando la extensión SQLTools:





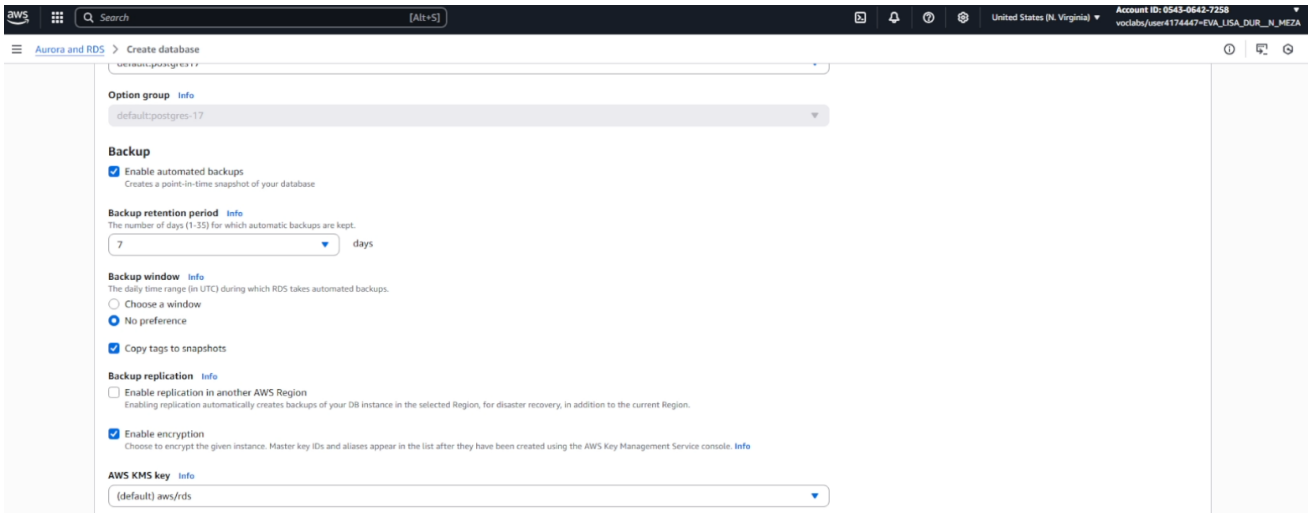
Una vez creada la base da datos y las tablas con sus respectivos registros se realizó la siguiente consulta como ejemplo.



4. Respaldo y restauración

Respaldo: es una copia de seguridad continua que AWS RDS gestiona de forma automática. Se activa al habilitar "Enable automated backups". RDS guarda cambios de la base y permite restaurar a un momento exacto dentro del período de retención. Incluye un almacenamiento gratuito igual al tamaño de tu base (más allá de eso, se cobra).

Al momento de crear la base de datos, se activó la opción de "Backup".



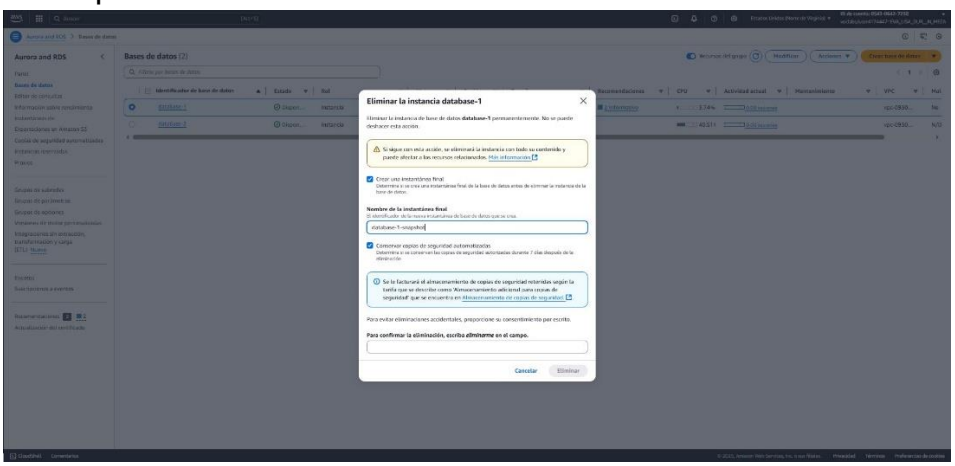
Snapshots: Son copias puntuales de la base de datos en un momento específico.

Tipos de Snapshots:

- ☑ Automáticos: los crea RDS según la política de backups que se configure (por ejemplo, retener por 7 días). Sirven para restaurar la base a un momento exacto dentro del período de retención.
- ☑ Manuales: Se crean cuando uno quiere. Se guardan indefinidamente (hasta que los borres) y son útiles antes de hacer cambios importante o actualizaciones de versión.

Los snapshots sirven para recuperar la base de datos en caso de fallo o pérdida de datos. Clonar una base para pruebas o desarrollo o migrar datos a otra región o cuenta de AWS.

Un ejemplo de Snapshot manual se da cuando se intenta eliminar la instancia RDS:



5. Métricas de monitoreo en CloudWatch

Se utilizarán las siguientes métricas de Amazon CloudWatch para monitorear la instancia de RDS con PostgreSQL:

Métrica	Justificación
CPUUtilization	Detecta sobrecarga de procesamiento. Un valor alto sostenido (>80%) puede indicar que la instancia no es suficiente.
DatabaseConnections	Controla la cantidad de conexiones activas. Si se acerca al límite máximo, puede causar errores de conexión.
FreeableMemory	Monitorea la memoria RAM libre disponible. Poca memoria disponible puede degradar el rendimiento.
ReadLatency / WriteLatency	Altas latencias afectan directamente la experiencia del usuario y reflejan problemas de E/S.
DiskQueueDepth	Número de operaciones esperando acceso al disco. Si es alto, puede indicar un cuello de botella.
ReadIOPS / WriteIOPS	Importante para analizar la carga de operaciones de entrada/salida por segundo.
SwapUsage	Uso excesivo de swap indica falta de memoria, lo que ralentiza la base de datos.

Estas métricas permiten anticipar problemas antes de que afecten la disponibilidad o escalabilidad del servicio.

6. Reflexiones sobre Escalabilidad

Si ShopNow continúa creciendo, implica que aumentan los usuarios y por tanto la cantidad de consultas que recibe la base de datos. Cada consulta consume recursos (como CPU, memoria, disco). Si muchas personas acceden al mismo tiempo esto puede aumentar los tiempos de respuesta, llevar a que el sistema se vuelva lento o colapse. Para evitar esto, se recomienda lo siguiente:

- **Aumentar la clase de instancia RDS** (por ejemplo, de db.t3.micro a db.t3.medium) para disponer de más CPU y memoria.
- **Activar almacenamiento escalable** (modo autoscaling de almacenamiento en RDS).
- **Habilitar read replicas**: o réplicas de lectura. Estas son copias sincronizadas de la base de datos principal, que están optimizadas solo para consultas de lectura. Se actualizan automáticamente cuando cambia el contenido de la base de datos original, no permiten escrituras (es decir, no se puede insertar, actualizar, ni eliminar directamente en ellas). Estas réplicas permiten distribuir la carga de trabajo cuando se tiene muchas lecturas recurrentes. Un ejemplo de esto podría ser el siguiente:

Tipo de nodo	Función principal	Usuarios que lo usan
Nodo principal	Escrituras: registrar pedidos, cambiar stock	Backoffice / lógica de negocio
Réplica 1	Lecturas: consultar productos	App pública (catálogo)
Réplica 2	Lecturas: historial de pedidos	App pública (perfil del usuario)

- **Implementar caching con Amazon ElastiCache**: este servicio de almacenamiento en memoria permite guardar temporalmente los resultados de consultas frecuentes. Funciona como un "cuaderno de respuestas rápidas". Por ejemplo, si muchos usuarios hacen la misma consulta, ElastiCache responde desde memoria. Esto permite reducir la presión sobre la base de datos porque se hacen menos lecturas a disco y la base de datos se concentra solo en operaciones críticas (como registrar un nuevo pedido).
- **Utilizar una arquitectura desacoplada**: Separar las cargas de trabajo (lectura, escritura, analítica) en diferentes servicios de AWS según sus necesidades.

7. Conclusiones

La migración de ShopNow hacia una base de datos gestionada en la nube (Amazon RDS con PostgreSQL) representa un avance estratégico con múltiples beneficios:

- **Facilidad operativa**: Reducción de tareas administrativas gracias al mantenimiento automatizado de AWS.
- **Escalabilidad**: Posibilidad de ajustar recursos según la demanda sin necesidad de infraestructura física.
- **Alta disponibilidad y recuperación**: Backups automáticos, snapshots y recuperación sencilla.
- **Seguridad**: Cifrado en tránsito y en reposo, control de accesos, monitoreo constante.
- **Cuidados**: Es importante monitorear costos para evitar cargos inesperados, y establecer alertas ante picos de consumo.

Pasos siguientes para la evolución de la infraestructura de ShopNow:

- Migrar el backend de la aplicación a contenedores (ECS o Fargate).
- Implementar una CDN (como CloudFront) para contenido estático.
- Integrar mecanismos de seguridad más avanzados como WAF o IAM roles para cada microservicio.

8. Referencias

https://keepcoding.io/blog/que-es-acid-bases-datos/
https://aws.amazon.com/es/compare/the-difference-between-acid-and-base-database/
https://docs.aws.amazon.com/es_es/AmazonElastiCache/latest/dg/WhatIs.html